Phase 5 — Concept Deep-Dives

Three concepts — the "three pillars" — each covered with mechanics, worked examples, and senior-level interview Q&A:

12. Metrics with Prometheus (pull model, histograms, RED, cardinality)
13. Centralized structured logging with Loki (the label-index bet)
14. Distributed tracing with OpenTelemetry (context propagation, sampling)

12. Metrics with Prometheus

The principle

A metric is a pre-aggregated number about your system, cheap enough to record on every event and to keep at high resolution: request counts, durations, queue depths, orders confirmed. Prometheus's model has two defining choices: it **pulls** (services expose a `/metrics` endpoint; Prometheus scrapes on its schedule), and it stores **labeled time series** (`http_request_duration_seconds_count{service="order-service", route="/orders", status_code="202"}`). Every service in this platform now exposes real prom-client metrics; Traefik and RabbitMQ expose their built-ins; Prometheus scrapes all of it every 10s.

The minute sub-concepts

Pull vs push. Pull means the monitoring system decides the schedule, target discovery is explicit (you can SEE what should exist), and a down target is *detectable* (`up == 0`) rather than silently absent. Push (StatsD/OTLP-push) suits short-lived jobs and serverless, where nothing lives long enough to be scraped. Pull also fails safe: a slow Prometheus degrades resolution, not the services.

The four metric types. *Counter* — monotonically increasing (requests, orders); only ever read through `rate()`. *Gauge* — goes up and down (queue depth, heap). *Histogram* — counts observations into cumulative buckets; enables percentile *estimation* server-side and aggregation across instances. *Summary* — client-side exact quantiles, but un-aggregatable across replicas; histograms won for a reason.

Histograms and `histogram_quantile`. Our latency histogram has buckets 5ms...5s. `histogram_quantile(0.95, sum by (le, service)(rate(..._bucket[5m])))` estimates p95 by linear interpolation *within* the bucket the quantile falls into — accuracy is bucket-resolution-bound. Two consequences seniors know: choose buckets around your SLO (a 250ms SLO wants buckets at 200/250/300, so violations are crisp), and NEVER average percentiles across services — you aggregate the underlying buckets, which is exactly what histograms make possible.

Cardinality is the whole game. Every unique label combination is a separate stored time series. `route` is safe only because we record the route *template* (`/orders/:id`, via Fastify's `routeOptions.url`) — recording raw URLs would mint a series per order UUID and melt the TSDB. The platform's label sets are bounded by construction: `status` in {CONFIRMED, FAILED, REJECTED}, `outcome` in {succeeded, failed} etc. Rule: labels must come from small, closed sets — never user IDs, order IDs, emails.

RED and USE. For request-driven services: **R**ate, **E**rrors, **D**uration — our histogram gives all three (count-rate, status_code>=500 rate, quantiles). For resources: **U**tilisation, **S**aturation, **E**rrors — that's the RabbitMQ and Node-runtime metrics (queue depth is saturation of the async path). Instrument RED at every service boundary, USE at every resource, and dashboards design themselves.

Default labels beat scrape labels. Each registry sets `service` as a default label in-process, so series are correctly attributed even if someone scrapes the endpoint ad hoc; the scrape config only labels infra targets that can't label themselves (Traefik, RabbitMQ).

What metrics can't do. They're aggregates — they tell you p95 degraded, never *which request* or *why*. That's the hand-off to traces (exemplars formalize it) and logs. Knowing which pillar answers which question is the observability skill.

Practical worked example

The instrumentation is ~15 lines per service. The histogram observation hook:

```
app.addHook('onResponse', async (req, reply) => {
  const route = req.routeOptions?.url ?? req.url;   // template, not raw URL!
  if (route === '/metrics' || route.startsWith('/health')) return;
  httpRequestDuration
    .labels(req.method, route, String(reply.statusCode))
    .observe(reply.elapsedTime / 1000);
});
```

And a domain counter at a business-meaningful transition (order saga):

```
await setStatus(evt.orderId, 'CONFIRMED');
ordersTerminalTotal.labels('CONFIRMED').inc();
```

Run the three demo orders, then in Prometheus (http://localhost:9090) query `sum by (status) (orders_terminal_total)` — you get the saga funnel as data: 1 CONFIRMED, 1 FAILED, 1 REJECTED. Then `histogram_quantile(0.95, sum by (le, service) (rate(http_request_duration_seconds_bucket[5m])))` shows per-service p95 — the exact query behind the Grafana panel.

10 senior interview questions

1. **Prometheus pulls; StatsD pushes. Argue both sides and pick for a

long-running microservice fleet.** (Pull: explicit target inventory, up-detection, monitoring controls load; push: works for short-lived jobs, NAT-friendly. Fleet of long-running services -> pull; add Pushgateway only for batch jobs, sparingly.)

2. **Why can't you average p95s from two replicas, and what does a histogram

change?** (Percentiles aren't linear — avg(p95a, p95b) is meaningless. Histograms ship the raw bucket counts, which ARE summable; you compute the quantile AFTER aggregating buckets.)

3. **A teammate adds `user_id` as a metric label. What happens and what do

you do instead?** (Cardinality explosion — one series per user, memory/ ingest blowup. Move per-user questions to logs/traces; keep labels from closed sets.)

4. **Design the bucket boundaries for a 300ms-SLO endpoint.** (Cluster

buckets around the SLO — e.g. 100/200/250/300/400/500ms/1s — so the SLO-violation boundary is a bucket edge and error budgets are exact, not interpolated.)

5. **What's the difference between `rate()` and `irate()`, and when does

each mislead?** (rate = per-second average over the window — smooth, good for alerting; irate = last two samples — spiky, good for fast dashboards, terrible for alerts. Both need the window >= 2 scrape intervals.)

6. **How do counters survive process restarts?** (They don't — they reset to

0. `rate()` detects monotonicity breaks and compensates, which is exactly

why you never graph raw counters, only rates/increases.)

7. **RED vs USE — instrument a message consumer.** (It has no HTTP requests:

RED maps to events consumed/sec, handler failures, handler duration; USE adds queue depth (saturation), prefetch utilisation, redelivery rate.)

8. **When metrics show p95 latency doubled, what CAN'T they tell you, and

what's your next hop?** (Not which requests, not why — aggregates lose the individual. Next hop: exemplars/traces filtered to the slow window, then logs for the guilty trace IDs.)

9. **Scrape interval is 10s. What alerting mistake does that constrain?**

(Any `rate()` window must be several× the interval, alerts can't detect sub-10s spikes, and `for:` durations shorter than a couple of scrapes flap. Resolution bounds everything downstream.)

10. **The gateway histogram and Traefik's metrics disagree on request

counts. Name three legitimate reasons.** (Traefik counts edge requests incl. redirects/TLS failures the gateway never sees; health checks excluded in one, not the other; retries/replays counted per-hop; and scrape phase differences within the window.)

13. Centralized Structured Logging with Loki

The principle

Logs answer "what exactly happened," and in a 16-container system reading per-container logs stops scaling immediately. Centralized logging ships every container's stdout to one queryable store. Loki's defining bet: **index only labels** (service, level), not log content. Content is stored compressed and grep'd at query time *within* the label-selected streams. This platform was accidentally ready for it: every service already logs single-line pino JSON — Promtail discovers containers over the Docker socket, parses the JSON, labels the stream, and pushes to Loki.

The minute sub-concepts

Structured logging is the precondition. `logger.info({ orderId, status }, 'order updated')` emits one JSON line with machine-parseable fields. The alternative — string interpolation — makes every downstream query a regex archaeology dig. The discipline was established in Phase 1; Phase 5 is where it pays.

Loki's label model vs full-text indexing. Elasticsearch indexes every token of every line — queries on anything are fast, but the index often exceeds the data, and ingest is expensive. Loki indexes only the small label set; storage is cheap chunks; queries pay at read time. The trade is economics: logs are write-heavy/read-rarely, so indexing everything optimizes the rare path. The corollary discipline is identical to Prometheus: **labels must be low-cardinality** (service, level — never orderId; orderId lives in the JSON body and is filtered at query time).

LogQL mirrors PromQL deliberately. `{service="order-service"} | json | orderId="..."` — select streams by label, then pipeline-parse and filter. Because label keys match the metrics labels (`service`), Grafana can pivot from a latency panel to that service's logs in one click — that shared label scheme IS the integration, and it was a choice, not luck.

Collection topology. One Promtail per host (here: one container) reading all container logs beats a logging library in every app: apps stay decoupled from the log pipeline, crashes can't lose buffered logs inside the app, and stdout remains the 12-factor contract. Promtail's `docker_sd_configs` discovers containers and reads logs via the Docker API — the relabel step maps `com.docker.compose.service` -> `service`.

Levels as pino numbers. pino emits `level` numerically (30=info, 40=warn, 50=error) — bounded cardinality, so it's safe as a label; the runbook notes the mapping. Filtering errors platform-wide: `{level="50"}`.

Retention and loss-tolerance. Logs are the most voluminous pillar; local config keeps filesystem chunks with no replication (`replication_factor: 1`) — fine for a laptop, and the exact knob you'd point at when asked "what changes in prod?" (object storage, retention/compactor, replication).

Practical worked example

The entire app-side change for Phase 5 logging is: nothing. The pipeline is config:

```
scrape_configs:
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
    relabel_configs:
      - source_labels: ['__meta_docker_container_label_com_docker_compose_service']
        target_label: 'service'
    pipeline_stages:
      - json: { expressions: { level: level, msg: msg } }
```

```
      - labels: { level: }
```

Place a FAILED order, then in Grafana -> Explore -> Loki:

```
{service=~"order-service|inventory-service"} | json | msg=~".*compensation.*"
```

Both halves of the compensation appear — the orchestrator enqueueing the release and inventory executing it — interleaved with timestamps, across two containers, from one query. That's the capability per-container `docker logs` can never give you.

10 senior interview questions

1. **Loki vs Elasticsearch for logs — argue the trade.** (ES: full-text

index, fast arbitrary search, heavy ingest/storage; Loki: label-only index, cheap ingest/storage, query-time grep. Logs are write-heavy and read-rarely -> Loki's bet usually wins on cost; ES wins for search-heavy use like security forensics.)

2. **What belongs in a Loki label vs the log body, and why?** (Labels:

low-cardinality stream identity — service, level, env. Body: everything per-event — orderId, userId, durations. High-cardinality labels shatter streams into millions of tiny chunks and destroy both ingest and query.)

3. **Why collect via a node agent reading stdout instead of an in-app

shipper library?** (Decouples app from pipeline, survives app crashes, zero app dependencies/config, uniform across languages, honors the 12-factor stdout contract; in-app shippers buffer in the failure domain they're reporting on.)

4. **Your logs are multi-line stack traces and they arrive as N separate

entries. What broke and how do you fix it?** (Line-based collection splits them; fix at source — single-line JSON with the stack in a field, as pino does — or configure multiline stages, which are fragile. Structured logging at source is the real fix.)

5. **How do you correlate a log line to the request that caused it across

services?** (Propagated correlation/trace ID stamped into every log line — with OTel active, inject traceId into the pino mixin; then LogQL filter `| json | traceId="..."` reconstructs the request story, and Grafana links logs <-> traces.)

6. **A teammate logs at info inside the per-message consumer hot path.

Costs?** (Log volume scales with throughput — ingest cost, noise burying signal, and possible event-loop pressure. Hot paths get debug-level or sampled logging; info marks state transitions.)

7. **Design log retention for this platform in prod.** (Tiered: 24-72h hot

in Loki for ops, 30-90d in object storage for audit/debug, compliance streams longer per policy; compactor + retention rules per tenant/stream; never 'keep everything hot forever'.)

8. **`docker logs` already exists. What specifically does centralization

add?** (Cross-container queries in one place, retention beyond container lifecycle — logs survive `compose down` —, label-based filtering, correlation with metrics/traces in Grafana, and access without Docker host access.)

9. **When is grep-at-query-time (Loki) unacceptably slow, and what's the

escape hatch?** (Needle-in-haystack over huge windows with weak label selectors; mitigations: better labels, structured filters early in the pipeline (`| json | field=...` prunes fast), recording the hot query

as a metric, or promoting that use-case to a real index.)

10. **Why must the logging `service` label equal the metrics `service`

label?** (Cross-pillar pivoting: Grafana panel -> Explore carries the label; alerts link to logs pre-filtered. Divergent naming breaks every correlated workflow — the label scheme is an API between pillars.)

14. Distributed Tracing with OpenTelemetry

The principle

One checkout touches the gateway, the order service, Postgres, RabbitMQ, inventory, and payment. Metrics aggregate it away; logs shatter it across containers. A **trace** reconstructs the single request: a tree of **spans** (each a timed operation with attributes), stitched across process boundaries by **context propagation** — a trace ID that travels inside HTTP headers and, crucially here, inside RabbitMQ message headers. Phase 5 wires this with ZERO application code: OpenTelemetry's Node auto-instrumentation is loaded via `NODE_OPTIONS=--import`, patches the libraries we already use (fastify, pg, ioredis, mongodb, amqplib, http), and exports OTLP to Jaeger.

The minute sub-concepts

Span mechanics. A span = operation name, start/end timestamps, parent span ID, trace ID, attributes (db.statement, messaging.destination...), status. The tree of parent-child edges IS the causal structure; Jaeger's waterfall is just that tree on a timeline. Gaps between a parent and its children are un-instrumented time — often the actual bug.

Context propagation is the hard part — and W3C `traceparent` is the answer. In-process, context rides AsyncLocalStorage; across HTTP, the `traceparent: 00-<traceId>-<spanId>-01` header; across the broker, the SAME header injected into AMQP message properties by the amqplib instrumentation and extracted by the consumer. That last hop is what makes this platform's demo special: the trace does not end at the 202 — it continues THROUGH RabbitMQ into inventory and payment. Async hops appear with producer-consumer **links/follows-from** semantics rather than strict parent-child timing.

Auto vs manual instrumentation. Auto-instrumentation (library patching at load time) yields the skeleton — every HTTP call, query, publish, consume — for zero code. Manual spans add business semantics (`span: reserve-stock`, attribute `order.id`). The pragmatic sequencing is exactly what this phase does: auto first, everywhere; manual only where questions remain. The `--import` register trick works because Node loads the hook before the app, so `require`/`import` of instrumented libraries returns patched versions.

Sampling. Tracing everything is expensive at scale. *Head sampling* decides at trace start (cheap, uniform — the default `parentbased_always_on` here, correct for a demo); *tail sampling* decides after completion (keep all errors and slow traces, drop boring ones — needs a collector buffering whole traces). The senior answer to "do you trace 100%?" is "locally yes; in prod, head-sample a few % plus tail-keep errors/outliers."

The exporter must fail soft. Services do not `depends_on` Jaeger; the OTLP exporter buffers, retries, and drops. Telemetry that can take down the system it observes is a design failure — this is also why OTEL_* env only arrives via the observability compose layer: remove the layer, and the SDK is never even loaded.

Traces complete the pillar triangle. Metrics say WHAT degraded (p95 up), traces say WHERE (the span that stretched), logs say WHY (the error detail, found via the traceId). Each pillar's weakness is another's strength; the shared keys (service label, trace ID) are the joints.

Practical worked example

The "code" is one line of env in docker-compose.observability.yml:

```
NODE_OPTIONS: "--import @opentelemetry/auto-instrumentations-node/register"
OTEL_SERVICE_NAME: "order-service"
OTEL_EXPORTER_OTLP_ENDPOINT: "http://jaeger:4318"
```

Place a CONFIRMED order, open Jaeger (http://localhost:16686), select `order-service`, find the trace. The waterfall reads like the saga's biography: gateway proxy span -> order-service POST /orders -> pg INSERT (order + outbox, same span cluster) -> amqplib publish `order.created` -> **inventory-service consume** (context crossed the broker!) -> pg UPDATE reserve -> publish `inventory.reserved` -> order consume -> publish `payment.requested`-side spans -> payment consume -> publish `payment.succeeded` -> order consume -> pg UPDATE CONFIRMED. One trace ID, five processes, both hops through RabbitMQ visible as producer->consumer edges. This single screenshot is the most persuasive artifact in the whole project.

10 senior interview questions

1. **Walk through how a trace ID crosses an async message broker.** (Producer

instrumentation injects `traceparent` into message headers at publish; consumer instrumentation extracts it before the handler runs and starts a span linked to the producer context — parent/link semantics, surviving arbitrary queue delay.)

2. **Auto-instrumentation vs manual — when is each wrong?** (Auto-only:

spans lack business meaning, noisy, misses in-process logic. Manual-only: enormous toil, inconsistent coverage, misses library internals. Auto for skeleton + targeted manual spans/attributes is the working answer.)

3. **What is `traceparent` and why did W3C standardization matter?**

(`version-traceId-parentSpanId-flags`; before it, B3/vendor headers fragmented propagation across mixed fleets — standardization made cross-vendor, cross-team propagation interoperable by default.)

4. **Head vs tail sampling — design for 'keep all errors, 1% of the

rest'.** (That policy is tail sampling by definition: a collector buffers spans until trace completion, applies policy; costs memory + a completion heuristic/timeout. Head sampling alone cannot see the error before deciding.)

5. **Your trace shows a 900ms gap between two spans with nothing in

between. Interpret it.** (Un-instrumented work: event-loop blockage, GC, queue wait before consume, connection-pool wait, or sync CPU. The gap's parent tells you which process to profile; often the answer is 'time in the queue' — check the broker consume timestamp.)

6. **Why must telemetry export never be on the request path, and how does

OTel ensure it?** (BatchSpanProcessor: spans buffered and exported async; bounded queues drop under pressure; exporter failures retry with backoff and then drop. Observability must degrade before it degrades the system.)

7. **How do traces, metrics, and logs reference each other in practice?**

(Exemplars attach trace IDs to histogram buckets; logs carry traceId via logger integration; shared service naming. Workflow: alert (metric) -> exemplar trace -> trace's logs.)

8. **What's the overhead of tracing and where does it bite first?**

(Per-span allocation + context propagation on every async boundary; hot-path services with many tiny spans suffer first; mitigations: sampling, span reduction, disabling noisy instrumentations — we disabled fs/dns-level noise via OTEL_NODE_ENABLED_INSTRUMENTATIONS.)

9. **A junior asks why the checkout trace's consumer spans aren't strict

children of the publish span. Explain.** (The consume may happen long after publish — strict parent-child timing would imply the producer 'contains' it. OTel models async hand-off as a link/follows-from: causal, not temporal containment.)

10. **You get one afternoon to add tracing to a 30-service Node fleet.

Plan it.** (Exactly Phase 5's play: OTLP endpoint + collector/Jaeger; roll out env-based auto-instrumentation (`--import` register) via the deploy layer — no code changes; verify propagation across one async hop; THEN iterate manual spans on the top user journeys. Instrumentation via configuration is the force multiplier.)